José Luis Riera López-Valverde

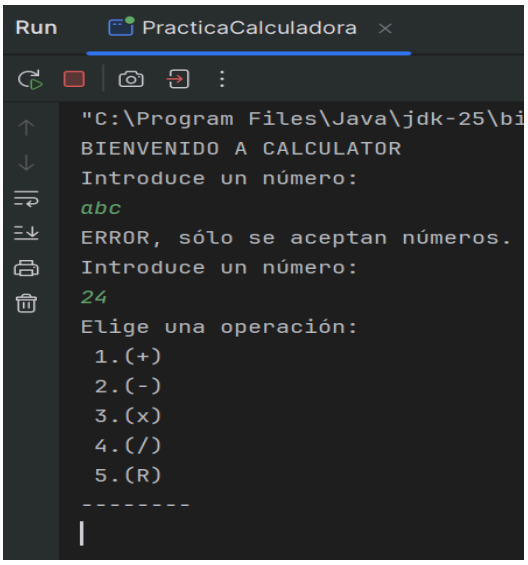
PRÁCTICA CALCULADORA

Introducción

Voy a presentar mi programa, que consiste en una calculadora interactiva. Su función principal es permitir al usuario realizar operaciones matemáticas básicas como sumas, restas, multiplicaciones, divisiones y raíces cuadradas. El programa pide al usuario que introduzca un número, luego el tipo de operación que desea hacer, y en algunos casos un segundo número para completar el cálculo. Además, he añadido controles de error para que el programa sea más seguro y no se bloquee si el usuario introduce texto en lugar de números, intenta dividir entre cero o calcula la raíz de un número negativo. De esta forma, la calculadora no solo realiza las operaciones, sino que también evita errores comunes durante la ejecución.

a) Prueba con texto y números negativos

Cuando el usuario introduce texto en lugar de un número, el programa detecta automáticamente que la entrada no es numérica gracias a la comprobación con `teclado.hasNextDouble()`.
En ese momento muestra el mensaje:

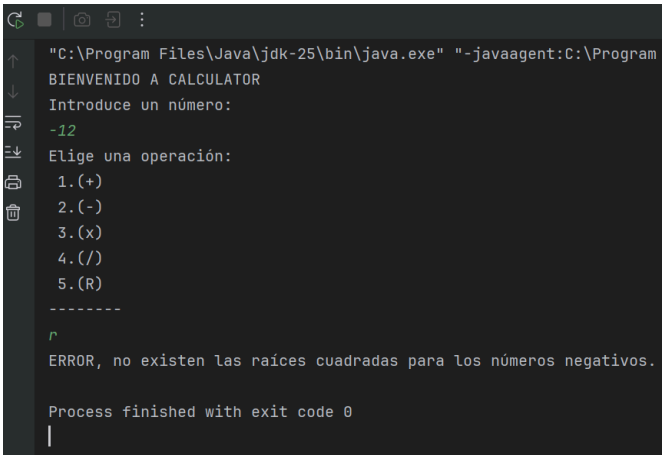


y vuelve a pedir el número hasta que el usuario introduzca un valor correcto.
El programa no se bloquea ni se detiene, sino que gestiona el error con un bucle `do-while` que se repite mientras el valor no sea válido.

En el caso de introducir números negativos y querer realizar una raíz cuadrada (R), el código incluye esta condición:

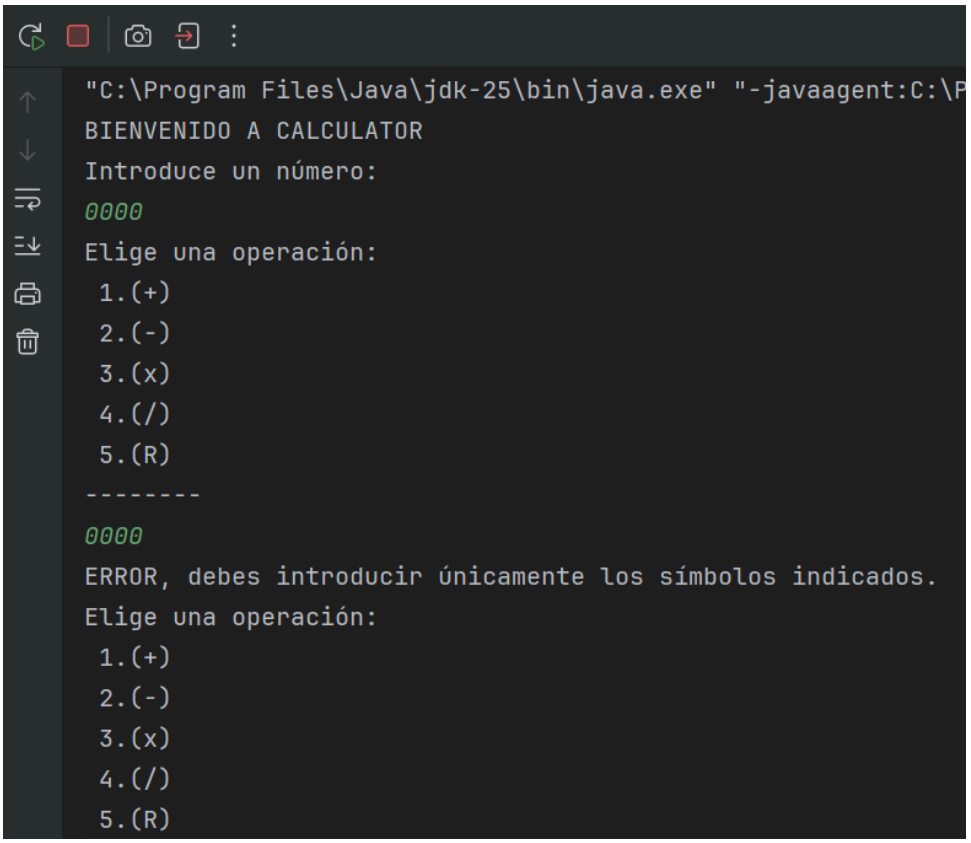
```
if (num1 > 0)
```

Por tanto, si el número introducido es negativo, el programa muestra el mensaje:



b) Prueba introduciendo ceros

Cuando se introduce ceros, lo acepta como el primer número, pero en la siguiente pregunta si pones ceros, no te dejará continuar porque no es uno de los signos que te pide.



c) Prueba con ceros en el denominador en el caso de la división

El programa está preparado para detectar divisiones por cero antes de realizarlas. Durante la división, el bloque `if` comprueba si el segundo número (`num2`) es 0:

```
if (num2 == 0) {
    System.out.println("Error, no se pueden dividir números entre cero.");
}
```

Por tanto, si el usuario intenta dividir entre 0, no se ejecuta la operación, y el programa muestra:

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\
BIENVENIDO A CALCULATOR
Introduce un número:
6
Elige una operación:
 1.(+)
 2.(-)
 3.(x)
 4.(/)
 5.(R)
--------
/
Introduce otro número para poder operarlo con el primero:
0
Comencemos a dividir.
Error, no se pueden dividir números entre cero.

Process finished with exit code 0
```

Esto evita que Java produzca un resultado infinito o una excepción de cálculo.
En cambio, si el divisor no es cero, la división se realiza normalmente y se muestra el resultado correcto.

d) Prueba cambiando el formato de entrada del signo

El programa ofrece varias operaciones posibles, y pide al usuario un símbolo para elegir la operación:
+, -, x, /, o R (para raíz cuadrada).

Por lo que si el usuario escribe una letra o símbolo no válido, aparece:

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\P
BIENVENIDO A CALCULATOR
Introduce un número:
234
Elige una operación:
 1.(+)
 2.(-)
 3.(x)
 4.(/)
 5.(R)
--------
j
ERROR, debes introducir únicamente los símbolos indicados.
Elige una operación:
 1.(+)
 2.(-)
 3.(x)
 4.(/)
 5.(R)
--------
d
ERROR, debes introducir únicamente los símbolos indicados.
```

CAMINO FELIZ:

```
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C
BIENVENIDO A CALCULATOR
Introduce un número:
1246
Elige una operación:
 1.(+)
 2.(-)
 3.(x)
 4.(/)
 5.(R)
--------
/
Introduce otro número para poder operarlo con el primero:
12
Comencemos a dividir.
RESULTADO = 103.83333333333333

Process finished with exit code 0
```